- **Epochs**: Number of complete passes through the training dataset.
- **Batch Size**: Number of samples processed at a time during training.
- **Validation Split**: Fraction of training data used for validation.

Step 6: Evaluate the Model

```
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

Visualizing Model Performance

Plot the training and validation accuracy and loss over epochs to assess the model's performance.

```
import matplotlib.pyplot as plt

# Plot training and validation accuracy
plt.plot(history.history['accuracy'], label='Training Accuracy') plt.plot(history.history['val_accuracy'],
label='Validation Accuracy') plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()

# Plot training and validation loss
plt.plot(history.history['loss'], label='Training Loss') plt.plot(history.history['val_loss'],
label='Validation Loss') plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

Enhancements to CNNs

1. **Data Augmentation**:
    - Augmenting the dataset by applying transformations such as rotations, flips, and zooms can increase model robustness.
2. **Transfer Learning**:
    - Leveraging pre-trained models such as VGG, ResNet, or Inception as a starting point can